

## **PROJECT: Cloud Infrastructure Automation using Terraform and AWS**

**Services:** EC2, RDS, VPC, Terraform

**Description:** Automate the provisioning of an application stack with Terraform. The stack includes EC2 instances in a VPC with a public and private subnet, a load balancer, and an RDS database.

**Skills:** Infrastructure as Code (IaC), Terraform, VPC setup

**Definition:** Cloud Infrastructure Automation using Terraform and AWS is the practice of provisioning and managing cloud resources through code instead of manual configuration. Using tools like Terraform with services from Amazon Web Services, infrastructure such as VPCs, EC2 instances, and databases can be created automatically. This approach follows the concept of Infrastructure as Code (IaC), enabling consistent and repeatable deployments. It reduces human errors and speeds up the setup process of complex cloud environments.

**Scope of the project:** The scope of the Cloud Infrastructure Automation using Terraform and AWS project is to design, deploy, and manage a secure and scalable cloud environment using Infrastructure as Code. It involves automating the creation of networking components like VPC, public and private subnets, and routing, along with compute resources such as EC2 instances and storage systems like RDS databases. Using Terraform with Amazon Web Services, the project covers end-to-end provisioning, configuration, and management of resources through code. It also includes setting up load balancing, security groups, and basic monitoring to ensure high availability and reliability. Overall, the project focuses on reducing manual effort, improving consistency, and enabling easy scaling and maintenance of cloud infrastructure.

## Methodologies:

1. Infrastructure as Code (IaC) – Using Terraform to define and manage infrastructure through code instead of manual setup.
2. Automation of Provisioning – Automating the creation of AWS resources like VPC, EC2, and RDS using Amazon Web Services.
3. Modular Configuration – Organizing Terraform files (main.tf, variables.tf, etc.) for better structure and reusability.
4. Version Control Practice – Managing infrastructure code changes systematically (commonly using tools like Git).
5. Declarative Approach – Defining the desired state of infrastructure, and Terraform automatically achieves that state.
6. Testing & Validation – Using commands like terraform validate and terraform plan to verify configurations before deployment.

## Services Used in this Project:

### 1. Amazon EC2 (Elastic Compute Cloud)

- Provides virtual servers in the cloud.
- Used to host and run the application.
- Acts as the main compute resource handling user requests.

### 2. Amazon RDS (Relational Database Service)

- Managed database service for MySQL, PostgreSQL, etc.
- Stores and manages application data securely.
- Runs inside private subnet for better security.

### 3. Amazon VPC (Virtual Private Cloud)

- Provides an isolated network environment in AWS.
- Controls communication between resources.
- Contains subnets, route tables, and gateways.

#### 4. Terraform

- Used to automate infrastructure creation using code.
- Manages AWS resources efficiently.
- Ensures consistent and repeatable deployments.

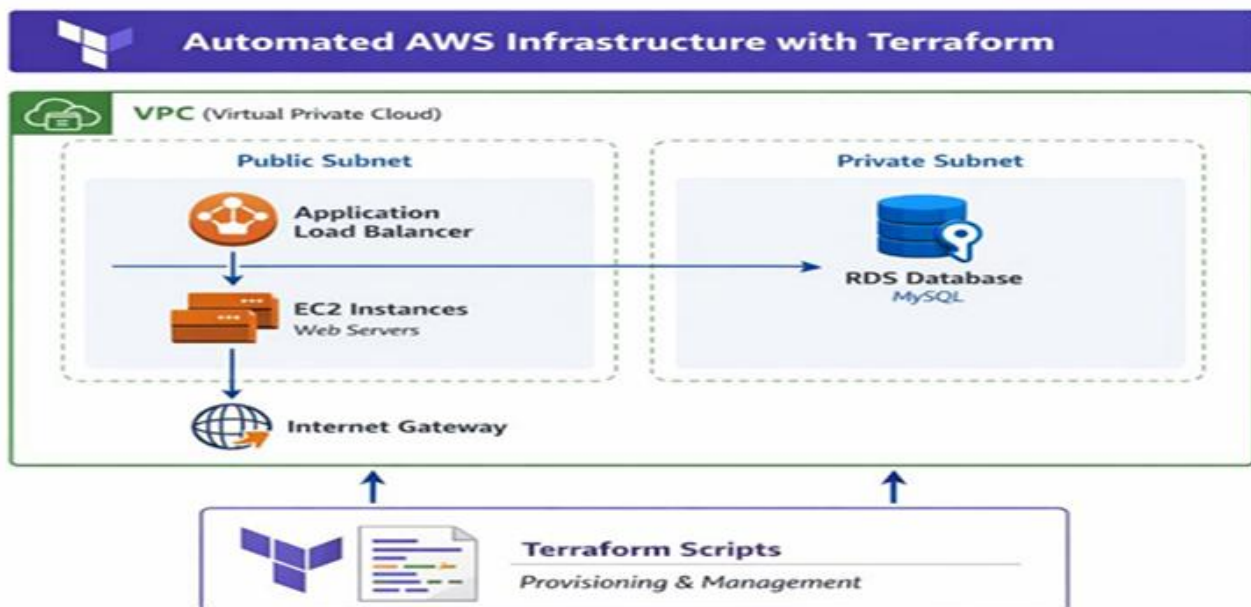
#### 5. Elastic Load Balancing (Application Load Balancer)

- Distributes incoming traffic across multiple servers.
- Improves availability and fault tolerance.
- Ensures better performance of the application.

#### 6. AWS Identity and Access Management (IAM)

- Manages user access and permissions.
- Provides secure authentication for AWS resources.
- Used for creating access keys for Terraform.

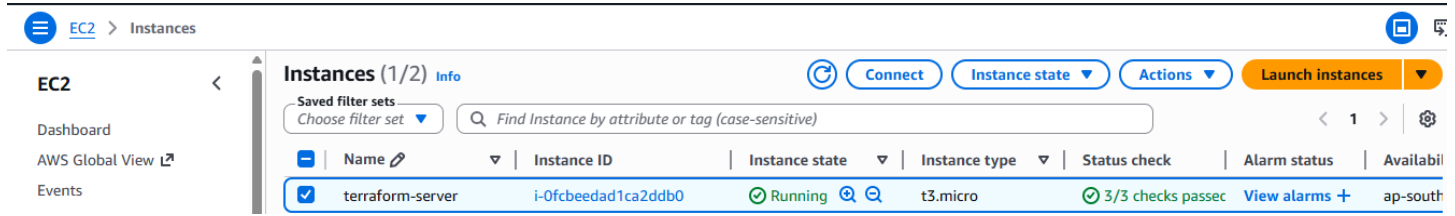
#### Architecture Diagram:



# Implementation and Execution of Cloud Infrastructure Automation using Terraform and AWS.

## STEP 1: Launch an EC2 Server

- Instance Type: t3. micro
- Volume: 20 GB
- Region: Mumbai (ap-south-1)
- Key Pair: Project-Key
- AMI: Amazon Linux 2023 kernel



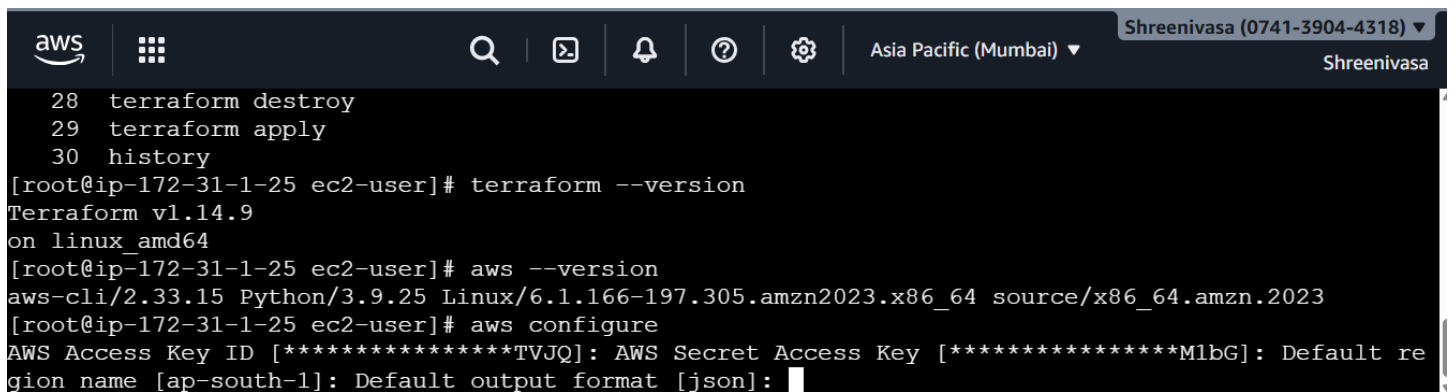
## STEP 2: Install and Check the Terraform version

```
[root@ip-172-31-1-25 ec2-user]# terraform --version
Terraform v1.14.9
on linux_amd64
[root@ip-172-31-1-25 ec2-user]#
```

## STEP 3: Check aws version

```
[root@ip-172-31-1-25 ec2-user]# aws --version
aws-cli/2.33.15 Python/3.9.25 Linux/6.1.166-197.305.amzn2023.x86_64 source/x86_64.amzn.2023
[root@ip-172-31-1-25 ec2-user]#
```

## STEP 4: Create Access Keys and Configure AWS CLI



## STEP 5: Create a directory

```
gion name [ap-south-1]: Default output format [json]: [root@ip-172-31-1-25 ec2-user]# cd terraform-aws-project/
[root@ip-172-31-1-25 terraform-aws-project]# ls
main.tf  outputs.tf  terraform.tfstate  terraform.tfstate.backup  terraform.tfvars  variables.tf
[root@ip-172-31-1-25 terraform-aws-project]#
```

## STEP 6: variable.tf

```
[root@ip-172-31-1-25 terraform-aws-project]# nano variables.tf
[root@ip-172-31-1-25 terraform-aws-project]#
```

```
variable "region" {
  default = "ap-south-1"
}
```

```
^G Help      ^O Write Out  ^F Where Is   ^R Cut        ^T Execute3 line^C Location  M-
U Undo      ^X ExitMark   M^R Read File ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-
E Redo      -6 Copy      B Where Was
```

## STEP 7: terraform.tfvars

```
[root@ip-172-31-1-25 terraform-aws-project]# nano terraform.tfvars
[root@ip-172-31-1-25 terraform-aws-project]#
```

```
aws | [Search] [Alt+S] | [Icons] | [Settings] | Asia Pacific (Mumbai) | Shreeni
GNU nano 8.3 terraform.tfvars
region = "ap-south-1"
```

## STEP 8: main.tf

```
[root@ip-172-31-1-25 terraform-aws-project]# nano main.tf
[root@ip-172-31-1-25 terraform-aws-project]#
```

A screenshot of a terminal window with a dark background. The top bar shows the AWS logo, a search bar, and the text "[Alt+S]". The right side of the bar shows "Asia Pacific (Mumbai)" and "Shreenivasa (0741-3904-4318)". The terminal title is "main.tf". The content of the file is Terraform configuration for an AWS VPC and subnets.

```
GNU nano 8.3 main.tf
provider "aws" {
  region = var.region
}

# ----- VPC -----
resource "aws_vpc" "main_vpc" {
  cidr_block = "10.0.0.0/16"
  tags      = { Name = "main-vpc" }
}

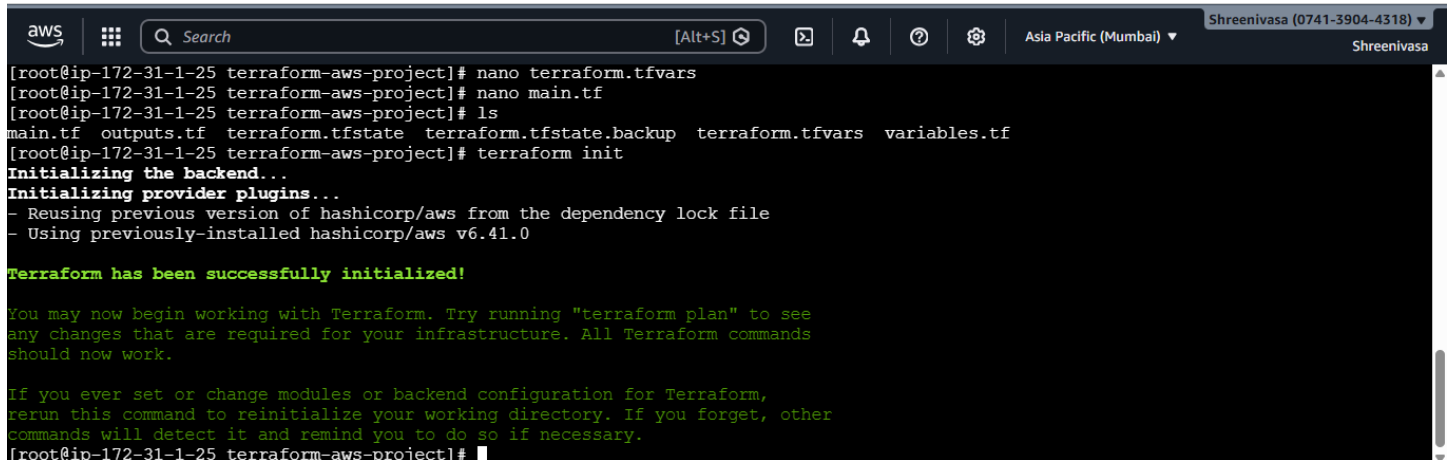
# ----- Public Subnets -----
resource "aws_subnet" "public_subnet_1" {
  vpc_id            = aws_vpc.main_vpc.id
  cidr_block        = "10.0.1.0/24"
  availability_zone = "ap-south-1a"
  map_public_ip_on_launch = true
  tags              = { Name = "public-subnet-1" }
}

resource "aws_subnet" "public_subnet_2" {
```

## STEP 9: Created a project directory and added four Terraform files

```
[root@ip-172-31-1-25 terraform-aws-project]# nano variables.tf
[root@ip-172-31-1-25 terraform-aws-project]# nano terraform.tf
[root@ip-172-31-1-25 terraform-aws-project]# nano terraform.tfvars
[root@ip-172-31-1-25 terraform-aws-project]# nano main.tf
[root@ip-172-31-1-25 terraform-aws-project]# ls
main.tf outputs.tf terraform.tfstate terraform.tfstate.backup terraform.tfvars variables.tf
[root@ip-172-31-1-25 terraform-aws-project]#
```

## STEP 10: Initialize Terraform

A screenshot of a terminal window showing the output of the 'terraform init' command. The top bar is similar to the previous screenshot. The terminal content shows the initialization process, including reusing the previous version of the AWS provider and successfully initializing Terraform.

```
[root@ip-172-31-1-25 terraform-aws-project]# nano terraform.tfvars
[root@ip-172-31-1-25 terraform-aws-project]# nano main.tf
[root@ip-172-31-1-25 terraform-aws-project]# ls
main.tf outputs.tf terraform.tfstate terraform.tfstate.backup terraform.tfvars variables.tf
[root@ip-172-31-1-25 terraform-aws-project]# terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.41.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[root@ip-172-31-1-25 terraform-aws-project]#
```

## STEP 12: Format Terraform Configuration

```
commands will detect it and remind you to do so if necessary.
[root@ip-172-31-1-25 terraform-aws-project]# terraform fmt
[root@ip-172-31-1-25 terraform-aws-project]#
```

## STEP 13: Validate Terraform Configuration

```
aws | Search [Alt+S] | Asia Pacific (Mumbai) | Shreenivasa (0741-3904-4318) | Shreenivasa

[root@ip-172-31-1-25 terraform-aws-project]# terraform init
Initializing the backend...
Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.41.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[root@ip-172-31-1-25 terraform-aws-project]# terraform fmt
[root@ip-172-31-1-25 terraform-aws-project]# terraform validate
Success! The configuration is valid.

[root@ip-172-31-1-25 terraform-aws-project]#
```

## STEP 14: Check Infrastructure Plan

```
aws | Search [Alt+S] | Asia Pacific (Mumbai) | Shreenivasa (0741-3904-4318) | Shreenivasa

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
[root@ip-172-31-1-25 terraform-aws-project]# terraform fmt
[root@ip-172-31-1-25 terraform-aws-project]# terraform validate
Success! The configuration is valid.

[root@ip-172-31-1-25 terraform-aws-project]# terraform plan
aws_vpc.main_vpc: Refreshing state... [id=vpc-012f8a283e2e2dd9c]
aws_internet_gateway.igw: Refreshing state... [id=igw-0e79d8c2446351a13]
aws_subnet.private_subnet_1: Refreshing state... [id=subnet-0cc6db0822b65a7f0]
aws_subnet.private_subnet_2: Refreshing state... [id=subnet-04ff3c6dcf19b706e]
aws_security_group.web_sg: Refreshing state... [id=sg-069793c3801052626]
aws_lb.target_group.tg: Refreshing state... [id=arn:aws:elasticloadbalancing:ap-south-1:074139044318:targetgroup/target-group/4cc647db46a47745]
aws_route_table.public_rt: Refreshing state... [id=rtb-01b2991160a6e60ea]
aws_subnet.public_subnet_2: Refreshing state... [id=subnet-08cce14d234995695]
aws_subnet.public_subnet_1: Refreshing state... [id=subnet-0bea42f26e53900e8]
aws_route.default_route: Refreshing state... [id=r-rtb-01b2991160a6e60ea1080289494]
aws_instance.web: Refreshing state... [id=i-02628a44fc2c6ee1b]
```

## STEP 15: Deploy Infrastructure

```
aws | Search [Alt+S] | Asia Pacific (Mumbai) | Shreenivasa (0741-3904-4318) | Shreenivasa

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
[root@ip-172-31-1-25 terraform-aws-project]# terraform apply
aws_vpc.main_vpc: Refreshing state... [id=vpc-012f8a283e2e2dd9c]
aws_lb.target_group.tg: Refreshing state... [id=arn:aws:elasticloadbalancing:ap-south-1:074139044318:targetgroup/target-group/4cc647db46a47745]
aws_subnet.public_subnet_1: Refreshing state... [id=subnet-0bea42f26e53900e8]
aws_subnet.private_subnet_2: Refreshing state... [id=subnet-04ff3c6dcf19b706e]
aws_internet_gateway.igw: Refreshing state... [id=igw-0e79d8c2446351a13]
aws_route_table.public_rt: Refreshing state... [id=rtb-01b2991160a6e60ea]
aws_subnet.public_subnet_2: Refreshing state... [id=subnet-08cce14d234995695]
aws_subnet.private_subnet_1: Refreshing state... [id=subnet-0cc6db0822b65a7f0]
aws_security_group.web_sg: Refreshing state... [id=sg-069793c3801052626]
aws_route_table.association.public_assoc_2: Refreshing state... [id=rtbassoc-0f283617327e50e7c]
aws_db_subnet_group.db_subnet_group: Refreshing state... [id=db-subnet-group]
aws_route_table.association.public_assoc_1: Refreshing state... [id=rtbassoc-0231f6496126a728f]
aws_lb.alb: Refreshing state... [id=arn:aws:elasticloadbalancing:ap-south-1:074139044318:loadbalancer/app/my-alb/bf30205a299be89a]
aws_route.default_route: Refreshing state... [id=r-rtb-01b2991160a6e60ea1080289494]
aws_instance.web: Refreshing state... [id=i-02628a44fc2c6ee1b]
aws_db_instance.db: Refreshing state... [id=db-VBQTXH6DOHVQLTESO5J4QKXQO4]
aws_lb_listener.listener: Refreshing state... [id=arn:aws:elasticloadbalancing:ap-south-1:074139044318:listener/app/my-alb/bf30205a299be89a]
```



```
aws_subnet.private_subnet_1: Refreshing state... [id=subnet-0cc6db0822b65a7f0]
aws_security_group.web_sg: Refreshing state... [id=sg-069793c3801052626]
aws_route_table_association.public_assoc_2: Refreshing state... [id=rtbassoc-0f283617327e50e7c]
aws_db_subnet_group.db_subnet_group: Refreshing state... [id=db-subnet-group]
aws_route_table_association.public_assoc_1: Refreshing state... [id=rtbassoc-0231f6496126a728f]
aws_lb_alb: Refreshing state... [id=arn:aws:elasticloadbalancing:ap-south-1:074139044318:loadbalancer/app/my-alb/bf30205a299be89a]
aws_route.default_route: Refreshing state... [id=r-rtb-01b2991160a6e60ea1080289494]
aws_instance.web: Refreshing state... [id=i-02628a44fc2c6ee1b]
aws_db_instance.db: Refreshing state... [id=db-VBQ7XKH6DOHVQLTESO5J4QKM2O4]
aws_lb_listener.listener: Refreshing state... [id=arn:aws:elasticloadbalancing:ap-south-1:074139044318:listener/app/my-alb/bf30205a299be89a/d3afe04e4d8a0c28]
aws_lb_target_group_attachment.tg_attach: Refreshing state... [id=arn:aws:elasticloadbalancing:ap-south-1:074139044318:targetgroup/target-group/4cc647db46a47745,i-02628a44fc2c6ee1b,80]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
[root@ip-172-31-1-25 terraform-aws-project]#
```

STEP 16: Verify Resources in AWS Console-----> EC2 (Elastic Compute Cloud)

EC2 > Instances

Dashboard

AWS Global View

Events

Instances

Instances

Instances (2/2) Info

Connect Instance state Actions Launch instances

Choose filter set Find Instance by attribute or tag (case-sensitive)

| <input checked="" type="checkbox"/> | Name             | Instance ID         | Instance state | Instance type | Status check    | Alarm sta |
|-------------------------------------|------------------|---------------------|----------------|---------------|-----------------|-----------|
| <input checked="" type="checkbox"/> | terraform-server | i-0fcbeedad1ca2ddb0 | Running        | t3.micro      | 3/3 checks pass | View alar |
| <input checked="" type="checkbox"/> | web-server       | i-02628a44fc2c6ee1b | Running        | t2.micro      | 2/2 checks pass | View alar |

STEP 17: VPC (Virtual Private Cloud)

VPC dashboard

AWS Global View

Filter by VPC

Virtual private cloud

Your VPCs

Subnets

Route tables

Your VPCs

VPCs VPC encryption controls

Your VPCs (3) Info

Find VPCs by attribute or tag

| <input type="checkbox"/> | Name          | VPC ID                | State     | Encryption c... | Encryption c |
|--------------------------|---------------|-----------------------|-----------|-----------------|--------------|
| <input type="checkbox"/> | terraform-vpc | vpc-02ca919dfcd5ffc9d | Available | -               | -            |

STEP 18: Public Subnet and Private Subnet

VPC dashboard

AWS Global View

Filter by VPC

Virtual private cloud

Your VPCs

Subnets

Route tables

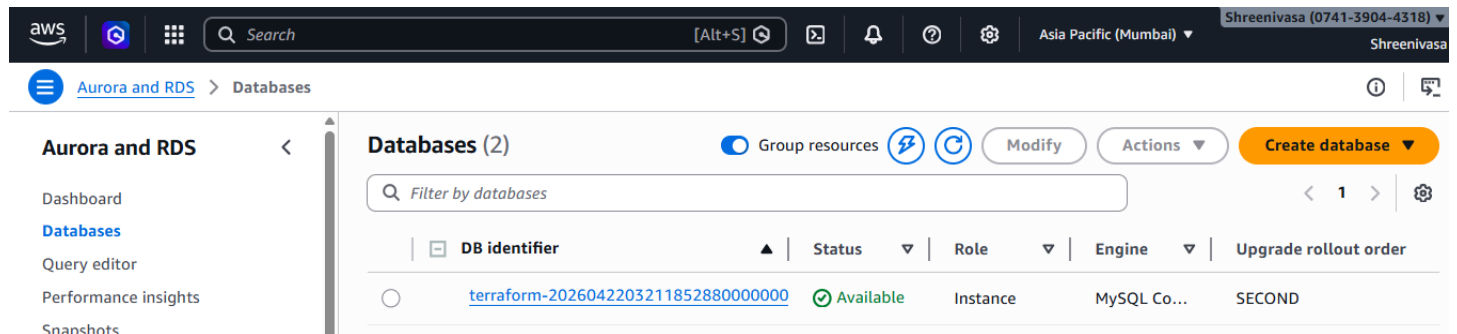
Subnets (8) Info

Find subnets by attribute or tag

| <input type="checkbox"/> | Name             | Subnet ID                | State     | VPC                          |
|--------------------------|------------------|--------------------------|-----------|------------------------------|
| <input type="checkbox"/> | private-subnet-1 | subnet-0cc6db0822b65a7f0 | Available | vpc-012f8a283e2e2dd9c   mair |
| <input type="checkbox"/> | private-subnet-2 | subnet-04ff3c6dcf19b706e | Available | vpc-012f8a283e2e2dd9c   mair |
| <input type="checkbox"/> | public-subnet-1  | subnet-0bea42f26e53900e8 | Available | vpc-012f8a283e2e2dd9c   mair |
| <input type="checkbox"/> | public-subnet-2  | subnet-08cce14d234995695 | Available | vpc-012f8a283e2e2dd9c   mair |



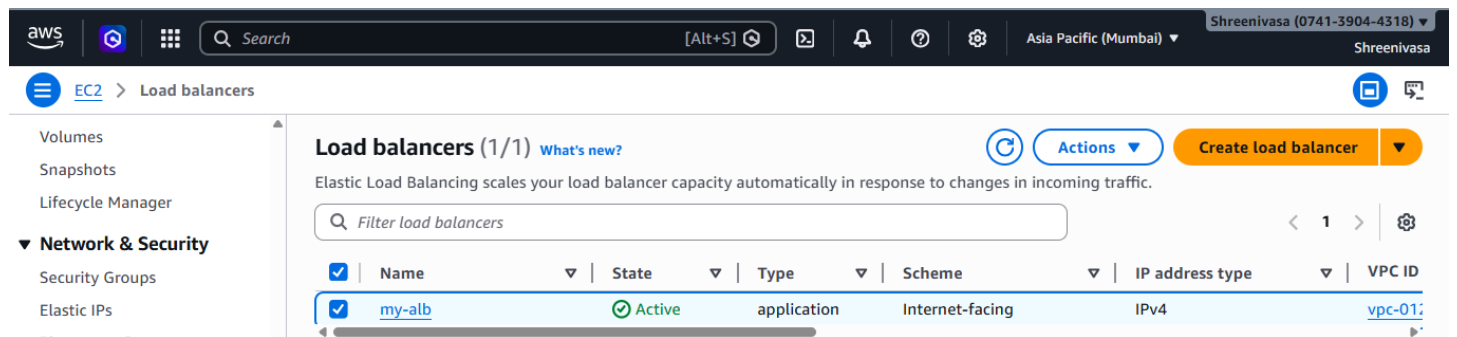
## STEP 19: RDS (Relational Database Management)



The screenshot shows the AWS Management Console interface for the 'Databases' section under 'Aurora and RDS'. The left sidebar contains navigation links: Dashboard, Databases (selected), Query editor, Performance insights, and Snapshots. The main content area shows 'Databases (2)' with a search bar and a table of database instances. The table has columns: DB identifier, Status, Role, Engine, and Upgrade rollout order. One instance is listed with the identifier 'terraform-2026042203211852880000000', status 'Available', role 'Instance', engine 'MySQL Co...', and rollout order 'SECOND'.

| DB identifier                                       | Status    | Role     | Engine      | Upgrade rollout order |
|-----------------------------------------------------|-----------|----------|-------------|-----------------------|
| <a href="#">terraform-2026042203211852880000000</a> | Available | Instance | MySQL Co... | SECOND                |

## STEP 20: LB (Load Balancer)



The screenshot shows the AWS Management Console interface for the 'Load balancers' section under 'EC2'. The left sidebar contains navigation links: Volumes, Snapshots, Lifecycle Manager, Network & Security (selected), Security Groups, and Elastic IPs. The main content area shows 'Load balancers (1/1)' with a search bar and a table of load balancers. The table has columns: Name, State, Type, Scheme, IP address type, and VPC ID. One load balancer is listed with the name 'my-alb', state 'Active', type 'application', scheme 'Internet-facing', IP address type 'IPv4', and VPC ID 'vpc-01...'.

| Name                   | State  | Type        | Scheme          | IP address type | VPC ID                    |
|------------------------|--------|-------------|-----------------|-----------------|---------------------------|
| <a href="#">my-alb</a> | Active | application | Internet-facing | IPv4            | <a href="#">vpc-01...</a> |

## Conclusion:

The Cloud Infrastructure Automation using Terraform and AWS project demonstrates how modern cloud environments can be efficiently built and managed using automation. By leveraging Terraform with Amazon Web Services, infrastructure components such as VPC, subnets, EC2 instances, load balancers, and RDS databases were provisioned in a consistent and repeatable manner. This approach reduces manual effort, minimizes configuration errors, and enables faster deployment of scalable and secure systems. The project also highlights the importance of Infrastructure as Code in improving reliability, maintainability, and version control of cloud resources. Overall, it reflects real-world DevOps practices and provides a strong foundation for building highly available and production-ready cloud architectures.